



**LANDMARK METROPOLITAN
UNIVERSITY INSTITUTE**
THE PRIDE OF AFRICA

DATABASE MANAGEMENT & SQL

Precious Oben, 2025/2026

LANDMARK DIGITAL LESSONS

(+237) 672 339 570 / 694 990 622 | www.landmark.cm



SQL TABLES



- Tables in RDBMS are collections of related data stored in rows and columns.
- When creating a table in a database, it is important to create the attributes or fields as a table cannot exist without attributes.
- An attribute or field is a characteristic or property which describes data held in a table.
- When setting fields of a table the datatypes of the fields must be defined as well as the constraints they hold.

SQL TABLES



- **Datatypes** define the kind of data a column can hold in a database table for example numerical data, text or dates etc.
- Some common datatypes include:
- CHAR: Fixed-length character string
- VARCHAR: Variable-length character string
- INT: Integer numeric value
- DECIMAL: Fixed-point numeric value
- DATE: Date value
- TIME: Time value

SQL TABLES



- **Constraints** are rules enforced on data in a database to ensure the accuracy, reliability, and integrity of the data stored within the database tables. Constraints are used to enforce the business rules on the data and can be specified when a table is created or altered.
- Constraints can be applied at column or table level.
- Column level constraints apply to a single column within a table. They enforce rules on the values that can be stored in that particular column.
- A key value of column-level constraints is that they ensure specificity. They are defined directly in the column definition. For example specifying a column cannot accept NULL values or has only UNIQUE values.
- Examples of column level constraints are PRIMARY KEY, UNIQUE, NOT NULL and CHECK.

SQL TABLES



- Table level constraints apply to the whole table, not just a single column.
- Table-level constraints are useful when the rule involves a relationship between different columns or even different tables.
- These constraints are defined separately from the column definitions, typically at the end of the table creation or alteration script.
- PRIMARY KEY, UNIQUE, NOT NULL and CHECK can also be defined at the table level. For example PRIMARY KEY can be defined on multiple columns or FOREIGN KEY which involves relationships can be defined at the table level.

SQL TABLES



- The general structure for defining datatypes and constraints is as follows:

```
CREATE TABLE table_name (  
    column_name1 datatype1 [constraint(s)],  
    column_name2 datatype2 [constraint(s)],  
    ...  
    [table-level constraint(s)]  
);
```


SQL TABLES



```
CREATE TABLE Employees (  
    ID INT PRIMARY KEY,  
    FirstName VARCHAR(100) NOT NULL,  
    LastName VARCHAR(100) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Email VARCHAR(255) UNIQUE,  
    StartDate DATE DEFAULT CURRENT_DATE  
);
```

SQL TABLES



```
CREATE TABLE EmployeeProjects (  
    EmployeeID INT,  
    ProjectID INT,  
    StartDate DATE,  
    EndDate DATE,  
    PRIMARY KEY (EmployeeID),  
    FOREIGN KEY (EmployeeID) REFERENCES Employees(ID),  
    CHECK (EndDate >= StartDate)  
);
```


INDEXES IN DATABASES



- Indexes in databases are special lookup tables that the database search engine can use to speed up data retrieval. Simply put, an index is a pointer to data in a table. An index in a database is very similar to an index in the back of a book.

Purpose of Indexes

- **Speed up retrieval of records:** Indexes make the query process faster by allowing the database system to find the row associated with a key value quickly.
- **Efficient data retrieval:** Without an index, the database system reads the whole table row by row to find relevant rows, which can be very inefficient for large tables.

Types of indexes

- **Single column indexes** – An index created on a single column
- **Composite indexes** – An index based on multiple columns
- **Unique indexes** – An index which ensures the rows have unique values
- **Full text indexes** - Used mainly for full-text searches in textual data columns.

INDEXES IN DATABASES



- Indexes are created using the “**CREATE INDEX**” statement or as part of column definition in CREATE TABLE or ALTER TABLE statements.
- The general syntax is shown below:

CREATE INDEX index_name ON table_name (column_name);

OR

CREATE INDEX index_name ON table_name (column1, column2);

INDEXES IN DATABASES



Uses of indexes in real world situations

- Improved customer experience
- Operational Efficiency
- Data Analysis and reporting

Drawbacks

- Choosing indexes
- Storage space
- Maintenance overhead

INDEXES IN DATABASES



- By default in most RDBMS, the query optimizer chooses whether to use an index or not when executing a query for different reasons.
- You can also explicitly suggest to the database engine to use a specific index for a query using the following hints :

SELECT * FROM table_name USE INDEX (index_name) WHERE condition;

SELECT * FROM table_name FORCE INDEX (index_name) WHERE condition;

SELECT * FROM table_name IGNORE INDEX (index_name) WHERE condition;

INDEXES IN DATABASES



To see created indexes:

SHOW INDEXES FROM table_name;

SHOW INDEX FROM table_name;

SHOW KEYS FROM table_name;

Assignment: Run any of the above codes after creating an index and give a description of the columns displayed.

TYPES OF DATABASE TABLES



Tables can be classified based on their longevity and storage mechanisms.

- **Temporary Tables:** They are very useful in situations where you need an intermediate result set for processing within a session. E.g.

```
CREATE TEMPORARY TABLE temp_table (  
    id INT AUTO_INCREMENT,  
    name VARCHAR(100),  
    PRIMARY KEY (id)  
);
```

TYPES OF DATABASE TABLES



- **Base Tables:** These are regular tables. They are persistent and store data across various sessions until explicitly dropped or altered. These tables can be accessed and manipulated by any user with the appropriate permissions.
- **Heap Tables (Memory tables):** These reside entirely in memory and are useful for quick lookups and are often used in environments where table content can be volatile, such as caching query results. The contents of memory tables are lost when the database server restarts.

```
CREATE TABLE memory_table (  
    id INT AUTO_INCREMENT,  
    value VARCHAR(255),  
    PRIMARY KEY (id)  
) ENGINE=MEMORY;
```

TYPES OF DATABASE TABLES



- **Partitioned tables:** They allow you to distribute parts of individual tables across a filesystem, according to rules which you can define based on columns used in the table. This is particularly useful for improving performance and management of large tables.

```
CREATE TABLE sales (  
    sale_date DATE NOT NULL,  
    product_id INT NOT NULL,  
    total_sales DECIMAL(19, 2)  
)  
PARTITION BY RANGE (YEAR(sale_date)) (  
    PARTITION p0 VALUES LESS THAN (1991),  
    PARTITION p1 VALUES LESS THAN (1992),  
    PARTITION p2 VALUES LESS THAN (1993),  
    PARTITION p3 VALUES LESS THAN (1994)  
);
```

TYPES OF DATABASE TABLES



Creating a temporary or base table to store the results of a query is a common practice in SQL, particularly when you want to perform complex calculations or transformations on data without affecting the original data set.

```
CREATE TEMPORARY TABLE IF NOT EXISTS DeptAvgSalaries AS  
SELECT department_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department_id  
HAVING COUNT(employee_id) > 5;
```


MODIFYING TABLES



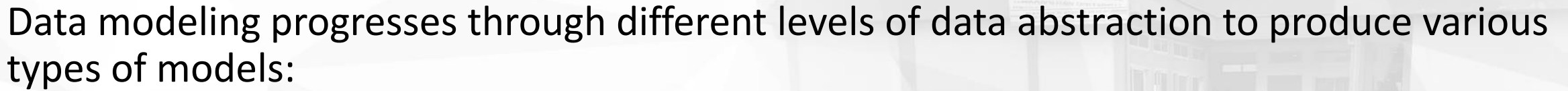
The UPDATE clause is used to make changes to entries in a table.

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```


DATA MODELING



- Data modeling is the process of creating a visual representation of an information system to show how data will be stored and used in a business operation. The primary goal of data modeling is to ensure that data objects offered by the functional team are represented accurately.
- Before modeling data, the requirements related to the information system must be known in order to know how to abstract data at each level.
- **Abstraction** refers to the process of reducing complexity by hiding unnecessary details and exposing only the essential features of a particular concept or system.



1. **Conceptual Data Model:** Focuses on the high-level structure of the data and the fundamental business rules, without considering technical implementations. It's used to outline the general data structure for the entire organization and is often the first step in database design, emphasizing the scope and definitions of entities and their relationships.

Its level of abstraction is the highest and can be called **conceptual level abstraction**. The audience for this model are business stakeholders and analysts. Its components include;

- Entities
- Attributes and
- Relationships

DATA MODELING



2. **Logical model** : Developed from the conceptual model by structuring data into database-specific schemas; involves decisions on the structure and constraints without considering the physical device or database system.

The level of abstraction here is medium and can be known as logical abstraction. The audience here includes Database designers and developers.

The components involved here are;

- Tables
- Columns
- Primary & Foreign keys
- Normalization

involved here are Database

le locations, clustering, etc.)

- up query responses
rformance

How These Models Are Achieved

- **Conceptual Data Model:** Created through discussions and interviews with business stakeholders to capture high-level business needs.
- **Logical Data Model:** Developed from the conceptual model by structuring data into database-specific schemas; involves decisions on the structure and constraints without considering the physical device or database system.
- **Physical Data Model:** Involves translating the logical model into a technical implementation that is optimized for the selected database management system.

- Conceptual models are represented by Entity – Relationship diagrams(ER). Logical and physical models are represented by more detailed ER diagrams and UML class diagrams. These diagrams are similar to ER diagrams but can include additional details such as methods (functions) that might be relevant to an application's data model. The physical model is implemented through SQL scripts.

Specialized Diagrams

- **EER (Enhanced Entity-Relationship) Diagram:** An extension of the basic ER diagram that includes more complex features like inheritance (subclasses and superclasses), categories (union types), and more detailed relationships (e.g., identifying relationships with optional participation). It can be used in both logical and physical modeling depending on the level of detail provided.

ER DIAGRAMS



- **Entity:** A real world object
- **Relation:** A table is a relation and consists of tuples(rows) and attributes (columns).
- **Schema:** the structural design or blueprint of a database that defines how data is organized and how the relations among them are associated. It outlines the tables, the fields in each table, and the relationships between fields and tables.
- **Relationship:** The association between two entities
- **Attribute:** Property that defines an entity

TYPES OF RELATIONSHIPS

- **One to one relationship (1:1):** This relationship occurs when one record in an entity is associated with one and only one record in another entity, and vice versa. E.g. A person owns an ID card and an ID card can only be owned by one person.
- **One to many(1:M) or Many to one relationship(N:1):** This is the most common relationship where a record in one entity can be associated with multiple records in another. The inverse is also true where many records in one entity relate to one record in another. E.g. A customer places many orders.

ER DIAGRAMS



- **Many to many relationship (M:N):** In this relationship, multiple records in one entity can be associated with multiple records in another entity. E.g. A student can enroll in many courses and each course can have many students.
- **Self-referencing relationships:** This occurs in within a single table. For example, in an Employees table, a manager and employee could both be represented in the same table with a foreign key linking an employee to their manager within the same table
- **Cardinality:** It is the numerical relationship between two entities in a relationship. It expresses the minimum and maximum number of instances of one entity that can or must be associated with each instance of the other entity. E.g. one-to-one, one-to-many, and many-to-many.
- **Ordinality:** refers specifically to whether the relationship is mandatory or optional from the perspective of both entities involved. It is sometimes represented by modality, which indicates the necessity of the relationship (e.g., a line ending in a circle means optional, and a line ending in a bar means mandatory).

SYMBOLS IN CONCEPTUAL ER DIAGRAMS

- **Entities:** Represented by rectangles.
- **Relationships:** Represented by diamonds.
- **Attributes:** Represented by ovals.
- **Primary Keys:** Underlined within the entity.
- **Cardinality:** Indicated by symbols near the entities, showing how many instances of one entity can be associated with one instance of another entity.
- **Ordinality (Optionality):** Shown by either a circle (optional) or a line (mandatory) near the entity on the relationship line.

ER DIAGRAMS IN WORKBENCH

- EER diagrams can be generated using the GUI by dragging and dropping and then mapped to a database schema or writing SQL scripts for generating tables and their relationships and running them.

